Python Programming
Assignment 1

Siddhesh Jadhav
Roll No: MS2510

September 3, 2026

GitHub Repository

GitHub Repository

1 Topics

1.1 Variables

What is it?
A variable is a name we give to a piece of data so we can store and refer to it later. In Python, we do not need to declare the type of a variable beforehand – Python figures out the type automatically based on the value assigned.

Why do we need it?
While a program runs, we often need to hold on to values – numbers, text, results of calculations – and reuse them later. Without variables, every value would have to be typed out again and again, which is impractical. Variables let us give data a name and access or update it whenever needed.

Where and How is it used?
Variables are used everywhere data needs to be held temporarily – in calculations, storing user input, loop counters, or saving the result of a function.

```
1 age = 21
2 name = "Siddhesh"
3 price = 99.5
```

Here `age`, `name`, and `price` are variables holding an integer, a string, and a float value respectively.

1.2 Basic Data Types

What is it?
A data type is a label that tells Python what kind of value a variable is holding. Python has several built-in data types, four of which are covered below.

Why do we need it?

The data type tells Python which operations are valid on a value (for example, addition works on numbers, concatenation works on strings). Using the wrong type for an operation leads to errors or incorrect results.

Where and How is it used?

Every variable has a type, which Python detects automatically (dynamic typing):

```
count = 5           # Integer
price = 99.5         # Float
is_active = True     # Boolean
name = "Siddhesh"    # String
```

Integer

An integer is a whole number – positive, negative, or zero – with no decimal point. It is used for counting, indexing, and loop iterations.

Float

A float is a number with a decimal point (e.g. 3.14). It is used for measurements, prices, and calculations that need precision.

Boolean

A boolean can only take two values: `True` or `False`. It is used to check conditions in `if-else` statements and loops.

String

A string is a sequence of characters (text data), written inside single or double quotes. It is used to represent names, messages, and file contents.

1.3 Arithmetic Operators

What is it?

Arithmetic operators are symbols used to perform mathematical calculations – such as `+, -, *, /, %, **, //`.

Why do we need it?

Any numerical calculation, from simple addition to computing averages, needs these operators. Without them a program could not process numbers at all.

Where and How is it used?

They are used everywhere from billing systems to statistics to game scoring:

```
a = 10
b = 3
print(a + b)   # addition -> 13
print(a - b)   # subtraction -> 7
print(a * b)   # multiplication -> 30
print(a / b)   # division -> 3.333...
print(a % b)   # modulus (remainder) -> 1
print(a ** b)  # power -> 1000
```

```
print(a // b)  # floor division -> 3
```

1.4 Comparison Operators

What is it?
Comparison operators compare two values and return `True` or `False` – such as `==, !=, >, <, >=, <=`.

Why do we need it?
Decision making – in `if-else` conditions and loop termination – requires comparing values against each other.

Where and How is it used?
Used inside conditions and loops:

```
x = 5
y = 10
print(x == y)   # False
print(x != y)   # True
print(x < y)    # True
print(x >= y)   # False
```

1.5 Assignment Operators

What is it?
Assignment operators are used to assign values to a variable – such as `=, +=, -=, *=, /=`.

Why do we need it?
They give a shorthand way to update the existing value of a variable, making code shorter and more readable.

Where and How is it used?
Used for updating counters and maintaining running totals:

```
count = 5
count += 1   # count = count + 1 -> 6
count -= 2   # count = count - 2 -> 4
count *= 3   # count = count * 3 -> 12
```

1.6 Type Conversion

What is it?
Type conversion means converting a value from one data type to another (e.g. string to integer).

Why do we need it?
User input often arrives as a string (for example, from `input()`), but a mathematical operation needs a numeric type. Conversion is required to bridge this gap.

Where and How is it used?
Done using built-in functions: `int(), float(), str(), bool()`

```
age_str = "21"
age_int = int(age_str)   # string -> integer
```

```
price = str(99.5)        # float -> string
```

1.7 Input and Output

What is it?
Input means receiving data from the user, and output means displaying results on the screen. Python provides `input()` and `print()` for this.

Why do we need it?
To make a program interactive, it must be able to take data from the user and show results back; otherwise the program would only work on fixed, hardcoded values.

Where and How is it used?
Used in forms, calculators, and virtually all interactive programs:

```
name = input("Enter your name: ")  # input from user
print("Hello", name)               # output display
```

Note: `input()` always returns a string, so it must be converted if a different type is needed.

1.8 Strings

What is it?
A string is an ordered sequence of characters, written inside quotes (`"text"` or `'text'`).

Why do we need it?
Strings are needed to represent text-based data such as names, messages, and file content.

Where and How is it used?
String Creation:

```
s = "Python Programming"
```

Indexing: Each character has a position number starting from 0; negative indexing counts from the end.

```
print(s[0])    # 'P'
print(s[-1])   # 'g'
```

Slicing: A portion of a string can be extracted using `[start:stop:step]`.

```
print(s[0:6])   # 'Python'
print(s[::-1])  # reversed string
```

String Operations: Concatenation (`+`), repetition (`*`), length (`len()`), etc.

```
greet = "Hello " + "Siddhesh"
print(greet * 2)
print(len(greet))
```

1.9 Lists

What is it?
A list is a data structure used to store multiple values in order inside a single variable,

written using square brackets `[]`. Lists are mutable, meaning their contents can be changed.

Why do we need it?

Lists are needed to group and manage related data together – for example, a list of marks or a list of names.

Where and How is it used?

Creating Lists:

```
fruits = ["Apple", "Banana", "Mango"]
```

Indexing:

```
print(fruits[0])    # 'Apple'
print(fruits[-1])   # 'Mango'
```

Slicing:

```
print(fruits[0:2])  # ['Apple', 'Banana']
```

Updating Elements: Since a list is mutable, a value can be changed directly using its index.

```
fruits[1] = "Orange"
```

Nested Lists: A list can contain another list inside it.

```
matrix = [[1, 2], [3, 4]]
print(matrix[0][1])   # 2
```

Basic List Methods:

```
numbers = [10, 20, 30]
numbers.append(40)    # add to end -> [10,20,30,40]
numbers.insert(1, 15) # add at a specific position
numbers.remove(20)    # delete a value
numbers.pop()         # remove last element
numbers.sort()        # ascending order
numbers.reverse()     # reverse the order
len(numbers)           # total number of elements
```